CAMP: Consumption-Aware Memory Prediction for Scientific Workflows

Nagarjuna Chowdary Kandimalla, Bhanu Prakash Vangala, and Tanu Malik
Department of Electrical Engineering and Computer Science
University of Missouri
Columbia, MO 65211, USA
{nkmh5@umsystem.edu, bv3hz@missouri.edu, tanu@missouri.edu}

***Abstract*—Distributed scientific workflow tasks exhibit significant variation in memory consumption, making accurate peak memory estimation critical for resource management and avoiding out-of-memory (OOM) failures. Existing workload schedulers rely on static features like file sizes or tool names, which fail to capture non-linear memory scaling. We present Consumption-Aware Memory Prediction (`CAMP`), a closed-loop framework that utilizes fine-grained, consumption-level provenance—the exact number of bytes a task reads—measured dynamically via eBPF kernel probes. To minimize tracing overhead across parallel task executions at scale, we develop an active-learning-based, multi-gate selective auditing policy that samples only the most informative workflow tasks for model retraining. We evaluate `CAMP` on thousands of production-scale bioinformatics workflow records executed under Slurm. Compared to size-based scheduling baselines, incorporating dynamic data-consumption features reduces prediction median absolute percentage error by up to 40.31%. Furthermore, `CAMP` successfully reduces predicted OOM events by 64.37% while slashing runtime-weighted memory wastage by 37.18%. Our selective auditing framework consistently outperforms random sampling across tight profiling budgets, proving that kernel-level consumption data provides a highly scalable and practical feedback signal for high-performance workflow resource allocation. A live Bowtie2 cold-to-warm evaluation reduces requested memory by 71.96% while retaining 99.78% first-attempt coverage.**

***Index Terms*—scientific workflows, memory prediction, eBPF, selective auditing, Slurm**

I. Introduction

Memory management remains a primary bottleneck in modern high-performance computing (HPC) and distributed cluster environments. Large-scale scientific workflows across domains such as genomics, remote sensing, and materials science routinely submit thousands of concurrent tasks to resource managers like Slurm [1], HTCondor [2], or Kubernetes [3]. Because cluster schedulers require declarative memory reservations prior to execution, accurate prelaunch resource estimation is critical. Underestimating memory requirements triggers out-of-memory (OOM) termination, causing cascading execution failures, costly checkpoint restarts, and broken dependencies across parallel pipeline stages [4], [5]. Conversely, overestimating requirements leads to substantial resource overprovisioning, low cluster utilization, and increased queuing delays for concurrent workloads [4], [6]. Because workflow developers cannot easily quantify the memory footprints of black-box analytical tools across highly variable datasets, reservations gravitate toward conservative upper bounds, resulting in cluster inefficiency [5], [6].

To automate resource scaling, machine learning models are increasingly deployed to predict task-level memory boundaries. Prior literature focuses predominantly on model selection, establishing that expressive non-linear architectures, such as k-nearest neighbors, random forests, and gradient-boosted trees—outperform traditional linear regressors at capturing complex runtime behavior [6], [7]. However, predictive accuracy remains fundamentally bounded by the available feature space rather than model capacity.

Current memory estimators suffer from two key information limitations. First, standard schedulers rely predominantly on static prelaunch features, such as staged input file sizes, which fail to reflect actual runtime execution patterns. Existing predictors use aggregate file byte volumes, process names, and thread counts to estimate peak allocation thresholds [4]–[6], [8]. However, static storage metrics do not map to the exact data subsets parsed during execution. For instance, selective query tools read minor fractions of a large file index, whereas memory-mapped binaries load file segments into physical memory via page faults without issuing conventional `read` system calls. Consequently, resource allocators conflate overall staged storage volume with live resident memory requirements. Resolving this discrepancy requires capturing fine-grained runtime data consumption, which is computationally expensive to trace continuously and causally unavailable at the moment of scheduling.

Second, streaming workloads display high variance in historical data density, creating an asymmetry in feature specificity. Frequently executed workflow steps possess dense historical records of nearly identical past runs, whereas novel or updated pipeline stages lack empirical baselines. Monolithic global models degrade under this variance: they overgeneralize well-characterized tasks by smoothing out tight, process-specific historical signals, yet generate over-optimistic or under-constrained bounds for unfamiliar tasks. An effective framework must isolate and route tasks dynamically according to the granularity of available empirical evidence.

This paper presents `CAMP`, a closed-loop memory-allocation framework designed to address these feature limitations. `CAMP` utilizes an eBPF-based auditing daemon to dynamically capture explicit read bytes and file-backed memory-mapped

(mmap) page faults, directly attributing these I/O operations to individual task process trees. Completed tasks populate an asynchronous database, supplying peak RSS records globally and kernel-level data consumption metrics for instrumented runs. To exploit historical data at the appropriate granularity, CAMP structures empirical observations into hierarchical compatibility *scopes*, ranging from specific instance repetitions to global hardware fallbacks.

To bound tracing overhead on production clusters, CAMP operates under a strict administrative profiling budget, deploying a multi-gate active learning selection policy. These gates prioritize tasks with a high probability of allocation failure, high anticipated training information utility, or those operating within under-observed feature regions. The resulting system evaluates a continuous loop of multi-quantile prediction, selective auditing, kernel measurement, and asynchronous retraining. The primary contributions of this work are:

- a *scope*-based hierarchical prediction architecture combining multi-scale history summarization and non-parametric quantile ensembles;
- an evaluation of fixed multi-gate selective auditing policies against uniform and process-stratified random baselines under equivalent sampling budgets;
- a comprehensive experimental evaluation on several Bioinformatics workflows including comparative performance baseline against Sizey [6] and a live end-to-end Bowtie2 case study evaluating the framework across cold-start, persistence, model updating, and execution phases.

To the best of our knowledge this is the first time a distribution based model is used for peak memory prediction. Past work [6] has predominantly considered point-based models.

The remainder of this paper is structured as follows. Section II formalizes the problem. Section III details the architectural design of CAMP, including the hierarchical scope definitions, history construction, multi-quantile inference, and active selection policies. Section IV defines the experimental workloads, evaluation protocol, baselines, and tracking metrics. Section V discusses the empirical results, Section VI covers related literature, and Section VII concludes the paper.

II. Problem Description

A workflow executes tasks $t = 1, \ldots, N$ arriving sequentially. Before launching, a predictive framework must allocate a memory estimate $\hat{y}_t$ to task t. Each task t is associated with:

- **Static features** ($\mathbf{s}_t$): A feature vector available at prelaunch scheduling time. This vector includes workflow metadata, process type, thread count, node constraints, including the aggregate staged input size $a_t = \sum_{f \in \text{staged}(t)} \text{size}(f)$.
- **Dynamic Consumption** (c_t): A scalar tracking the empirical byte volume consumed by the task during runtime execution. While highly predictive of peak memory requirements, this metric is observable exclusively through intrusive kernel-level instrumentation, rendering it causally hidden at task launch time ($c_t \notin \mathbf{s}_t$).

TABLE I
Notation used in the problem formulation.

Symbol	Description
$t = 1, \ldots, N$	Sequential task index
$\mathbf{s}_t$	Static features, known at launch
a_t	Aggregate staged input size
c_t	Dynamic consumption signal, instrumentation-only
y_t	True peak memory (max RSS)
$\hat{y}_t$	Predicted peak memory
$\mathcal{H}_m(t)$	History of prior peak-memory outcomes
$\mathcal{H}_c(t)$	History of prior audited consumption traces
$q_t \in \{0, 1\}$	Auditing decision for task t
$B \in (0, 1]$	Profiling budget (max audited fraction)
f	Memory predictor
π	Auditing policy
$L(y_t, \hat{y}_t)$	Asymmetric loss (underestimation penalized more)

- **Target Peak Memory** (y_t): The maximum Resident Set Size (RSS) physically measured post-execution.

Because tasks run concurrently and finish asynchronously, only information from tasks completed before this task's launch is available. We denote this causal history by two feature vectors, constructed only from information available before launch of task t:

- $\mathbf{h}_{m,t}$: features derived from peak-memory outcomes y_τ of tasks completed before launch of task t.
- $\mathbf{h}_{c,t}$: features derived from consumption traces c_τ of tasks completed *and audited* before launch of task t.

We write $\mathbf{h}_t = [\mathbf{h}_{m,t}, \mathbf{h}_{c,t}]$ for the combined history. Which prior tasks contribute to these vectors, and how their outcomes are summarized, is a modeling choice addressed in Section III.

At launch, a binary decision $q_t \in \{0, 1\}$ determines whether task t is to be instrumented:

- $q_t = 1$: the task is audited, so (c_t, y_t) becomes available to future $\mathcal{H}_c$ and $\mathcal{H}_m$.
- $q_t = 0$: only y_t becomes available; c_t is never observed.

Auditing incurs overhead, so the fraction of audited tasks is capped by a budget $B \in (0, 1]$:

$$\frac{1}{N} \sum_{t=1}^{N} q_t \leq B.$$

We seek a memory predictor f and an auditing policy π,

$$\hat{y}_t = f(\mathbf{s}_t, \mathbf{h}_t), \qquad q_t = \pi(\mathbf{s}_t, \mathbf{h}_t),$$

that jointly minimize cumulative prediction loss subject to the budget constraint:

$$\min_{f,\pi} \sum_{t=1}^{N} L(y_t, \hat{y}_t) \quad \text{s.t.} \quad \frac{1}{N} \sum_{t=1}^{N} \mathbb{E}[q_t] \leq B \tag{1}$$

L is asymmetric: it penalizes underestimation ($\hat{y}_t < y_t$) more heavily than overestimation, reflecting that under-provisioning risks OOM aborts while over-provisioning only wastes reserved resources.

III. CAMP DESIGN

We first describe how CAMP predicts $\hat{y}_t$, then (Section III-G) the gates that determine the auditing policy π.

CAMP is designed around two core empirical observations. First, although a task's true data consumption c_t cannot be observed by the scheduler before launch, it can be estimated using available prelaunch features and memory metrics. This lightweight, learned proxy $\hat{c}_t$ serves as a critical supporting signal for peak memory estimation.

Second, accurate memory allocation requires learning at the correct level of task specificity. Frequently repeated pipeline steps often possess dozens of nearly identical historical runs to learn from, whereas a newly deployed workflow stage may have none. Monolithic global models degrade under this variance by averaging over these non-uniform historical densities.

Central to the design of CAMP is a hierarchical system of *scopes*. A *scope* is a task compatibility criterion defined by task instance, staged input, structural features, workflow name, process type, software version, and underlying hardware profile. Scopes are used to isolate pools of prior completed tasks whose peak-memory and consumption histories are relevant to a pending execution. CAMP defines K distinct scopes, $I_0, \ldots, I_{K-1}$, ordered strictly from most specific (e.g., exact task instance previously executed) to most general (e.g., matching cluster hardware profile). A task uses exactly one active scope based on historical support, or a cold-start state when no scope is supported, but its feature space remains enriched by historical summaries across all scopes.

A. The Predictor Pipeline

The problem formulation seeks a single allocation function $f(\mathbf{s}_t, \mathbf{h}_t)$. CAMP instantiates f as a prediction pipeline: a global point model, process-specific upper-tail models with broader fallbacks, an auxiliary consumption estimator g, and a tail calibration layer $\mathcal{K}$. The pipeline executes sequentially:

$$\hat{c}_t = g(\mathbf{s}_t, \mathbf{h}_{m,t}) \tag{2}$$

$$\hat{y}_t = \mathcal{K}\Big(f\big(\mathbf{s}_t,\, \mathbf{h}_t,\, \hat{c}_t\big),\, \mathbf{s}_t,\, \mathbf{h}_t\Big). \tag{3}$$

The proxy $\hat{c}_t$ is computed from prelaunch-legal inputs $(\mathbf{s}_t, \mathbf{h}_{m,t})$ alone, so the pipeline uses no information beyond the inputs admitted by the problem statement. The asymmetric allocation objective L is implemented through quantile pinball loss and allocation-policy selection. The prediction models estimate the point and upper-tail quantiles of y_t, while the calibration layer $\mathcal{K}$ (Section III-E) selects a base request and corrects residual under-coverage.

B. Scope Hierarchy and Routing

Let $\mathcal{P}(t)$ denote the causal universe of all tasks that have fully completed prior to the launch epoch of task t.

For each hierarchy tier $i \in \{0, \ldots, K-1\}$, let $\mathcal{C}_{I_i}(t) \subseteq \mathcal{P}(t)$ represent the specific pool of completed tasks that satisfy scope i's compatibility criteria, and let n_i represent the minimum sample support threshold required to activate that tier.

A task is automatically routed to the strictest possible scope whose historical pool satisfies its support threshold:

$$\ell(t) = \min\big\{\, i \in \{0, \ldots, K-1\} : |\mathcal{C}_{I_i}(t)| \geq n_i \,\big\} \tag{4}$$

The final baseline fallback tier, I_{K-1}, uses matching system history and requires one completed task. If no tier satisfies its support threshold, the task enters a cold-start state and receives an administrative bootstrap default reservation.

The selected scope, or the cold-start state, is recorded as the task's history_scope and supplied as a model input feature. It does not partition the peak-memory models or restrict adjacent historical pools from contributing multi-scale statistical features to the broader history vector $\mathbf{h}_t$. The calibration layer evaluates the same hierarchy independently over residual history.

C. Proxy Consumption Estimator

CAMP trains a single consumption model g, global across scopes but necessarily restricted to *audited* tasks, since c_τ is observed only when $q_\tau = 1$. Its training set is:

$$\{(\mathbf{s}_\tau, \mathbf{h}_{m,\tau}) \mapsto c_\tau : q_\tau = 1\} \tag{5}$$

Note that g conditions on the memory history $\mathbf{h}_{m,t}$ only, matching the inputs available at both training and inference time. Pooling audited tasks across all scopes gives g substantially more training data than any individual process. During development, its out-of-fold prediction $\hat{c}_t$ is appended as an input feature to the peak-memory models; during inference, the fitted global model supplies the same feature. The predictive accuracy of g therefore depends directly on *which* tasks the auditing policy selects, which we discuss in Section III-G.

D. Peak-Memory Quantile Models

CAMP fits a global Quantile LightGBM (QLGBM) ensemble for the point prediction and independent upper-tail ensembles for each workflow-process pair. Each process model is trained exclusively on rows belonging to that pair. During online learning, a workflow model provides a broader upper-tail fallback when available, followed by an independently trained global fallback. The regression target is the true post-execution peak memory footprint y_t, optimized under pinball loss across a vector of target quantiles $\mathcal{Q} = \{0.50, 0.90, 0.95, 0.99, 0.995\}$:

$$f : \big(\mathbf{s}_t, \mathbf{h}_t, \hat{c}_t\big) \longmapsto \{Q_q\}_{q \in \mathcal{Q}} \tag{6}$$

where Q_q represents the predicted q-quantile of peak memory. To minimize variance and stabilize predictions, each individual quantile target is modeled by an ensemble of three models initialized with distinct seeds, and the median value is selected. A sorting pass is executed immediately following inference to eliminate quantile crossing.

E. Inference and Tail Calibration

During online execution, the global proxy model first appends $\hat{c}_t$. The global point model and the most specific available upper-tail model then output the ordered quantile set. A workflow process model is preferred, followed by the global fallbacks.

Because scientific workload memory footprints are highly heavy-tailed, CAMP considers candidate base requests from the point prediction, nearest-neighbour peak history, and upper-tail predictions. Each candidate is bounded below by the point prediction. On development tasks, CAMP selects the candidate that satisfies the global and per-workflow process coverage requirements with the smallest request-to-peak ratio. The selected candidate becomes the base request b_t.

CAMP then processes b_t through a calibration layer $\mathcal{K}$ that adjusts the request based on the multiplicative scale-free log-residuals of past allocations:

$$r_\tau = \log(y_\tau / b_\tau) \tag{7}$$

A positive log-residual indicates a historical model under-prediction. Residual records are aggregated continuously per scope. Let $\widehat{Q}_\alpha(\cdot)$ denote the empirical quantile operator evaluated over a target residual set, where α is selected with the base request on development tasks. The calibration layer selects the strictest residual scope that satisfies its support threshold and blends its residual profile with a broader parent tier $p(i)$:

$$\rho_t = \eta_i \, \widehat{Q}_\alpha\big(r_i\big) + \big(1 - \eta_i\big) \, \widehat{Q}_\alpha\big(r_{p(i)}\big) \tag{8}$$

where I_0, I_1, and I_2 use I_3 as the parent tier, I_3 uses I_4, and I_4 and I_5 use I_5. The weight η_i is a local residual confidence weight derived from scope sample support and log-residual variance; the I_2 weight also includes nearest-neighbour distance. The final calibrated memory reservation request $\hat{y}_t$ is scaled exponentially and rounded upward to the nearest whole MiB boundary:

$$\hat{y}_t = \lceil b_t \exp(\max(0, \rho_t)) \rceil_{\mathrm{MiB}} \tag{9}$$

F. History Construction: $\mathbf{h}_{m,t}$, $\mathbf{h}_{c,t}$

Both models above condition on summarized history rather than raw task-level records. For every scope $i \in \{0, \ldots, K-1\}$—not just $i = \ell(t)$—CAMP computes a summary block, so a task routed to a strict scope still sees broader, more stable statistics as context. Each block summarizes the peak-RSS values (and, where audited, consumption values) of scope i's pool into medians, upper-tail percentiles, spreads, and sample support. Exact field definitions, confidence metrics, and imputation mechanics are detailed in the accompanying technical report [9].

G. Selective Auditing

Full eBPF auditing supplies the consumed-data signal that extends $\mathbf{h}_{c,t}$, but auditing every task incurs the overhead measured in Section V. The auditing policy π therefore decides, before launch, which pending tasks should spend the budget B. An audit cannot help the task that produces it i.e., its value begins only when the record becomes history for later tasks.

Each pending task is scored from an audit-feature vector $\mathbf{x}_t$ drawn from the same prelaunch-legal information available to the allocator: $\mathbf{s}_t$, $\mathbf{h}_t$, the active request, and the model's quantiles. The task's own y_t, c_t, runtime, and outcome are excluded; they appear only as labels when scoring models are trained on completed tasks. Because score scales differ across workflows, the model outputs and discovery signals are converted to within-workflow percentile ranks $\mathrm{rank}_w(\cdot) \in (0, 1]$ over pending tasks of the same workflow, while history confidence enters directly.

The budget is dominated by a single question: *is the active memory request likely to be too small?* Tasks whose requests are about to fail are valuable audit targets because they concentrate consumed-data observations exactly where the predictor was weakest. CAMP estimates this with two lightweight models trained on completed tasks: a classifier for the underallocation probability $p_t^r = \Pr(y_t > \hat{y}_t \mid \mathbf{x}_t)$, and, fitted only on completed underallocated tasks, a regressor for the conditional deficit $\hat{\delta}_t = \mathbb{E}[y_t - \hat{y}_t \mid y_t > \hat{y}_t, \mathbf{x}_t]$. The *risk score*

$$R_t = 0.70 \ \mathrm{rank}_w(p_t^r) + 0.30 \ \mathrm{rank}_w(p_t^r \, \hat{\delta}_t) \tag{10}$$

ranks a task highly when failure is likely, when its expected deficit is large, or both. At each decision point, CAMP derives an exact audit count from $B\,|U|$, divides it among workflows in proportion to pending counts, and spends 80% of each workflow's quota on its highest-R_t tasks.

The remaining 20% assigns 10% to a *heavy-information* score selecting tasks whose consumed-data measurement is likely to improve future predictions and splits the final 10% between *discovery* and random draws from workflow-process groups. The random component protects against unmonitored drift and keeps inclusion probabilities non-degenerate; score definitions and quota mechanics appear in the technical report [9].

IV. Experimental Method

All results are available at [10].

A. Workflows, Datasets, and Platform

The matched offline cohort contains 33,516 task records from six workflows (Table II). Nextflow/nf-core [11], [12] runs on AWS ParallelCluster with Slurm and 13 worker nodes, each with 32 vCPUs and 256 GiB RAM.

GATK and Minimap2 are deterministic custom workloads constructed for this study. GATK generates paired reads over hg38 for repeated HaplotypeCaller scatters, while Minimap2 generates ONT-style reads over fixed genomic windows and runs mapping, indexing, and summary steps. Sarek uses GIAB NA12878 30x WGS with GRCh38 interval scattering. Taxprofiler uses 5,998 paired-end Illumina metagenomic WGS runs from an ENA snapshot. Seqinspector and Viralmetagenome use 3,000 runs from the same ENA cohort with the Kraken2 Standard-8 database. Together, these workflows cover variant

TABLE II
MATCHED WORKFLOW COHORT.

Workflow	Version	Dataset	All	Test	Proc.
GATK	4.5.0.0 custom	Synthetic hg38 reads	500	75	1
Minimap2	custom	ONT-style reads	6,001	900	5
Sarek [13]	3.6.0	GIAB NA12878 WGS	6,012	902	25
Seqinsp. [14]	1.1.0	ENA metagenomes	6,001	900	3
Taxprof. [15]	2.0.1	ENA paired-end WGS	5,999	900	2
Viralmeta. [16]	1.1.3	ENA metagenomes	9,003	1,349	6
Total	–	–	33,516	5,026	–

TABLE III
ACCESS AND CAMP MODEL INSTANTIATION.

Input parameters	Model outputs
Access: $\mathbf{x}_t^{\mathrm{Access}}$	$\{Q_q^{\mathrm{Access}}(t)\}_{q\in\mathcal{Q}}$ and calibrated request $\hat{y}_t^{\mathrm{Access}}$
CAMP: $\mathbf{x}_t^{\mathrm{CAMP}}$	$\{Q_q^{\mathrm{CAMP}}(t)\}_{q\in\mathcal{Q}}$ and calibrated request $\hat{y}_t^{\mathrm{CAMP}}$

calling, read alignment, taxonomic profiling, and metagenomic classification, producing different relationships between staged input size, measured consumed data, and peak memory.

B. CAMP Model Instantiation

The memory-allocation experiment evaluates two conditions on the same tasks. *Access* is the peak-history baseline: it uses the launch-time static vector and causal peak-memory history. *CAMP* is the consumption-aware allocator: it additionally uses causal consumption history from earlier audited tasks and a prelaunch estimate of the current task's consumption. Their legal input vectors are

$$\mathbf{x}_t^{\mathrm{Access}} = [\mathbf{s}_t, \mathbf{h}_{m,t}], \tag{11}$$

$$\hat{c}_t = g(\mathbf{s}_t, \mathbf{h}_{m,t}),$$

$$\mathbf{x}_t^{\mathrm{CAMP}} = [\mathbf{s}_t, \mathbf{h}_{m,t}, \mathbf{h}_{c,t}, \hat{c}_t]. \tag{12}$$

Thus, neither condition can observe the current task's measured c_t or y_t before launch. Table III maps these inputs to the outputs produced by each condition using the notation from Section III.

For each condition, the global point model and the matching workflow-process upper-tail model produce the quantile estimates listed in Table III; an unseen workflow-process pair uses the global upper-tail fallback. The calibration procedure in Section III-E evaluates the prediction and peak-history candidates and converts the selected base into the submitted memory request. Both conditions use the same task rows, peak-RSS target, quantile model family, and leakage-control protocol; each is fitted and calibrated using development data only. The next subsection describes this common model-training and calibration procedure.

C. Training and Calibration Protocol

The 33,516-task cohort is divided chronologically, with the earlier 28,490 tasks used for development and the latest 5,026 tasks reserved for held-out evaluation. Related task-instance, staged-input, and sample groups remain in the same partition. Access and CAMP use the same development and test tasks, peak-RSS target, model family, and preprocessing. Their legal inputs are defined above, and each condition is fitted and calibrated independently. Peak RSS is modelled as $\log(1+y_t)$, with y_t expressed in MiB, while the consumption estimator uses $\log(1+c_t)$, with c_t expressed in bytes.

Five-fold grouped out-of-fold prediction is performed on the development tasks. Consequently, each development prediction is produced by models that were not trained on that task or its associated split group. These out-of-fold predictions support calibration and policy selection without exposing a row to its own target during prediction. Automated checks enforce identifier uniqueness, deterministic order, feature formulas, system compatibility, and exclusion of current post-run fields.

A deterministic task-identifier hash separates the out-of-fold development predictions into 17,210 residual-reference rows and 11,280 policy-selection rows. The residual-reference rows construct the condition-specific residual distributions used for calibration. Candidate allocation policies are then evaluated on the policy-selection rows. Among candidates satisfying at least 99% overall coverage and 97.5% coverage for every workflow, each condition selects the policy with the smallest request-to-peak ratio.

After policy selection, the models are refitted on all 28,490 development tasks, and their parameters and selected policies are frozen before held-out evaluation. The 5,026 test tasks are processed sequentially. A task's own outcome is unavailable when its request is produced; once revealed, it may update causal history and residual state for later tasks. No test outcome is used to refit model weights or reselect the allocation policy. The offline replay approximates recorded completion order, whereas live CAMP admits only outcomes completed before decision time d_t.

Together, these controls support four research questions"

RQ1: eBPF and mmap signal. Compared with STRACE, how much explicit-read and memory-mapped page-fault activity does eBPF capture, and what runtime overhead does online full auditing introduce?

RQ2: Consumption-aware allocation. Does CAMP reduce counterfactual first-attempt underallocation and runtime-weighted unused memory relative to Access, and what point-error and reservation-cost tradeoffs result?

RQ3: Comparison with Sizey. On identical offline task assignments, how does CAMP compare with unchanged Sizey in underallocation, point error, and reserved-memory cost?

RQ4: Selective auditing. At equal eBPF audit counts, do CAMP's gates identify more underallocated, memory-heavy, and consumption-informative tasks than random selection?

RQ5: Live Bowtie2 feedback. In fresh cold and warm executions, can CAMP allocate memory before each task, learn only from completed observations, and reduce warm-run reservation cost while maintaining high first-attempt coverage?

CAMP trains a global consumption ensemble and, for each condition, a global point ensemble, workflow-process upper-tail ensembles, and a global upper-tail fallback. Each peak-memory quantile uses three fixed-seed LightGBM models with the Access and CAMP inputs in Equations 11 and 12. Scope-pool construction dominates cost: key-matched scopes are $O(N)$, while I_2's nearest-neighbor search requires indexing at scale. At inference, CAMP evaluates the consumption estimator, global point model, and matching workflow-process upper-tail model, using the global upper-tail fallback for an unseen pair.

D. Evaluation Protocol and Metrics

For first request $\hat{y}_t$, recorded peak RSS y_t, q50 prediction $\tilde{y}_t$, and runtime τ_t, the common allocation metrics are

$$u_t = \mathbf{1}[\hat{y}_t < y_t], \qquad \mathrm{Cov} = 1 - \frac{1}{N}\sum_t u_t,$$

$$\kappa = \frac{\sum_t \hat{y}_t}{\sum_t y_t}, \qquad W = \sum_t \tau_t (\hat{y}_t - y_t)_+,$$

$$\mathrm{MdAPE} = 100 \operatorname*{median}_t \frac{|\tilde{y}_t - y_t|}{y_t}.$$

Here u_t marks a counterfactual underallocation, κ is the aggregate request-to-peak ratio, and W is unused memory-time, reported in GiB-hours. We also report total requested memory.

RQ1. Matched no-audit, full-STRACE, and full-eBPF executions report explicit-read and mmap-fault signal shares, absolute mmap-fault bytes, and runtime overhead $100(T_w^a/T_w^0 - 1)$ relative to no audit.

RQ2. Access and CAMP are compared using held-out underallocation counts across chronological grouped 30/70, 50/50, 70/30, 85/15, and 90/10 splits. Seeds 1996–2000 refit all models and policies at each split. For CAMP, underallocations per 1,000 tasks, the five-seed mean, t-based 95% confidence interval, and coefficient of variation measure seed sensitivity on identical holdout rows. For the primary 85/15 split, total requested memory and W are also reported.

RQ3. Sizey and CAMP receive identical tasks under Sizey's 9,141/21,337-task per-process split. Sizey's process-size eligibility rule removes approximately 3,000 records. Underallocation count, coverage, q50 MdAPE, κ, and W are reported, with κ and W normalized to Sizey.

RQ4. Similarly 9,141-task development set selects and freezes the three-gate policy before replaying 21,337 later tasks. At budget B, recall of target z_t is

$$\mathrm{Recall}_z(B) = \frac{\sum_t q_t(B) z_t}{\sum_t z_t}. \tag{13}$$

Risk-target and high-information-target recall are compared with uniform and process-stratified random selection at equal counts. Budgets of 5%, 10%, 20%, 30%, and 50% use 1,000 repetitions; random-baseline shading gives empirical 95% intervals.

RQ5. Two fresh 6,000-task Bowtie2 executions use the same workflow structure, inputs, worker pool, CAMP allocation policy, and retry policy. The cold run starts without Bowtie2 history; the warm run starts from the terminal model and completed-history state of the cold run. The runs use separate Nextflow work directories, full eBPF auditing, and no `-resume`. We report aggregate observed peak and first-requested memory, first-attempt coverage, unused memory-time, allocation-margin buckets, successful tasks, and recorded first-attempt OOMs.

V. Results

A. RQ1: Audit Signal and Cost

Figure 1(a) places workflows on the y-axis and the percentage composition of the eBPF consumed-data signal on the x-axis. As Fig. 1(a) shows, mmap faults contribute 50.7% of Minimap2's consumed-data signal and 16.9–19.9% for GATK and Viralmetagenome, demonstrating that explicit reads alone can miss substantial consumption. Figure 1(b) shows that eBPF full-audit overhead is not uniformly lower than STRACE: it is similar or lower for GATK, Sarek, and Taxprofiler, but higher for the other workflows. Thus, eBPF's main benefit is visibility into mmap-backed consumption, while its variable full-audit cost motivates selective auditing.

B. RQ2: Consumed-Data Feedback

Figure 2a(a) places the chronological development/holdout split on the x-axis and mean underallocations per 1,000 held-out tasks on the y-axis. Figure 2b(b) uses the same axes but shows CAMP's five training seeds as colored points; diamonds and error bars report the five-seed mean and 95% confidence interval, while CV is the coefficient of variation.

As Fig. 2b(a) shows, CAMP underallocates fewer tasks at 50/50 and 85/15, but more at 30/70, 70/30, and 90/10. Panel (b) shows that 85/15 has the greatest seed variation (CV 67.4%), whereas 70/30 is stable but consistently less safe. In the primary 85/15 run (seed 1996), CAMP reduces underallocations by 64.37% and unused memory-time by 37.18%, while increasing total requested memory by 31.50%. Thus, consumed-data feedback helps in some development regimes, but its present policy is not split-independent.

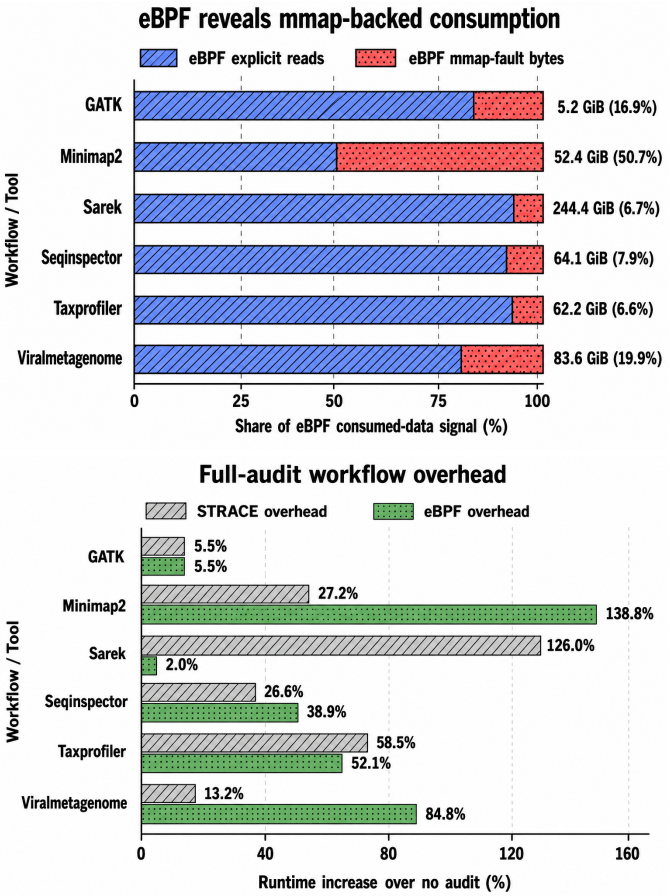


Fig. 1. RQ1 full auditing. (a) eBPF explicit-read and mmap-fault composition; labels give mmap bytes and signal share. (b) STRACE and eBPF workflow-runtime overhead over no audit.

C. RQ:3 External Comparison with Sizey

Figure 3(a) places Sizey and CAMP on the x-axis and the number of underallocated tasks on the y-axis. Panel (b) uses the same x-axis and reports median absolute percentage error on the y-axis. Panel (c) places request-to-peak ratio and unused memory-time on the x-axis and reports each value relative to Sizey on the y-axis; Sizey is normalized to 100%.

As Fig. 3(a) shows, CAMP reduces underallocations from 3,597 to 287, a 92.02% reduction, and raises first-attempt coverage from 83.14% to 98.65%. Panel (b) shows median absolute percentage error falling from 27.04% to 16.14%, a 40.31% improvement. Panel (c) shows the capacity tradeoff: CAMP's request-to-peak ratio is 225.5% of Sizey's and its unused memory-time is 141.4%. Thus, CAMP substantially improves safety and point accuracy, but uses more reserved capacity than Sizey.

D. RQ:4 Selective Auditing

Figure 4 places the audit budget, expressed as the percentage of tasks audited, on the x-axis and target recall on the y-axis. Panel (a) reports recall of risk targets whose active memory request would be smaller than peak RSS. Panel (b) reports recall of high-information targets meeting the joint memory-tail and consumption-utility label defined by the heavy-information gate. All policies select the same number of tasks at each budget.

In Fig. 4(a), CAMP recalls 15%, 25%, 43%, 57%, and 83% of risk targets at 5%, 10%, 20%, 30%, and 50% budgets, respectively, compared with 5%, 10%, 20%, 30%, and 50% for uniform random selection. Panel (b) shows high-information recall of 7%, 13%, 26%, 40%, and 58%, again above both equal-count random baselines at every displayed budget. Thus, the gates concentrate audits most strongly on underallocation risk while also finding more tasks useful for refreshing consumption-aware models.

E. RQ5: Live Cold-to-Warm Feedback

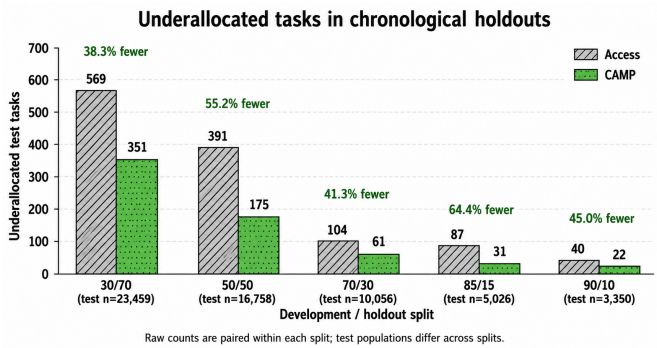
Figure 5(a) compares aggregate observed peak memory with the memory requested before each task outcome was known. The cold and warm runs execute 6,000 logical Bowtie2 tasks each and have nearly identical aggregate observed peaks: 238.50 and 238.82 GiB, respectively. In contrast, aggregate first-requested memory falls from 898.39 GiB in the cold run to 251.93 GiB in the warm run, a 71.96% reduction. The warm request is only about 5.5% above aggregate observed demand, whereas the cold request is approximately 3.77 times the observed peak.

Panel (b) weights excess requested memory by task runtime. Unused memory-time falls from 23.954 GiB-hours to 0.018 GiB-hours, a 99.93% reduction. This three-order-of-magnitude decrease shows that feedback from the completed cold run removes most of the reservation cost for the subsequent fresh execution, without relying on workflow-cache reuse.

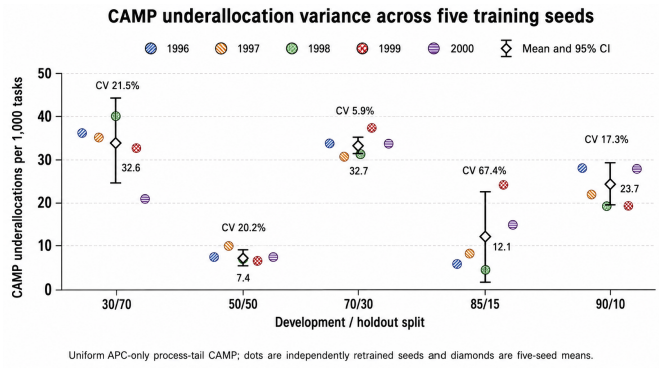
Panel (c) explains the change at task granularity. The share of tasks requested within 10% above their observed peak rises from 75.25% to 92.73%, while the share requested more than 25% above peak falls from 16.73% to 0.27%. The tighter warm allocations introduce a small safety tradeoff: first requests below observed peak increase from 4 to 13 tasks because the warm run makes tighter memory requests, leaving less buffer for prediction error, so first-attempt coverage changes from 99.93% to 99.78%. Overall, the cold-to-warm experiment demonstrates that completed Bowtie2 outcomes can substantially tighten later memory reservations while retaining very high first-attempt coverage.

VI. Related Work

Workflow memory prediction. Prior work estimates task requirements from prelaunch inputs and completed tasks [17]. Ferreira da Silva et al. [18], [19] update runtime, disk, and peak-memory estimates as tasks complete; Tovar et al. choose historical allocations or reshape splittable analyses [8], [20]; and Witt et al. optimize allocation cost using online feedback [4], [21]. Sizey trains and reselects multiple models through resource-allocation quality [6]; Ponder selects task-sizing strategies [5]. Related methods predict time-varying memory [22], learn allocation actions [23], profile runtime cold starts [24], or learn from scheduler logs [7], [25]. CAMP



(a) RQ2 underallocations with different splits.



(b) RQ2 allocation variance across repeated evaluations.

Fig. 2. RQ2 allocation safety and robustness. (a) Held- out underallocation results. (b) Variation across repeated evaluations.

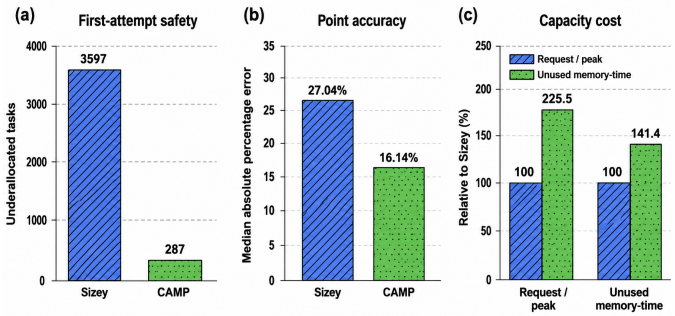


Fig. 3. 21,337-task comparison with Sizey: (a) underallocations; (b) median absolute percentage error; (c) request/peak and unused memory-time normalized to Sizey.

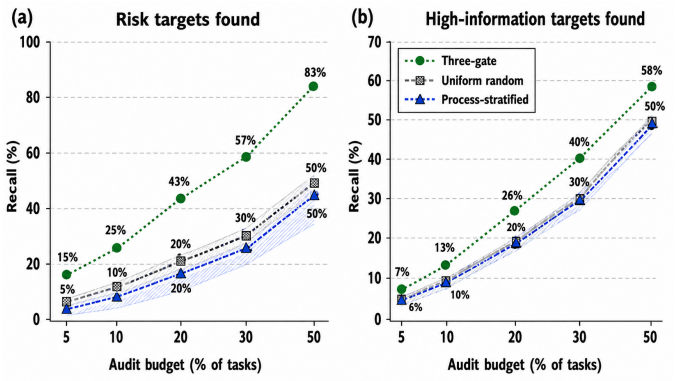


Fig. 4. RQ4 equal-count selective-audit recall. Curves are means over 1,000 repetitions; shading gives empirical 95% random-baseline intervals and the dotted diagonal is budget-proportional recall.

instead changes allocator information by separating staged size from selectively measured consumption and admitting only completed history. Its external experiment preserves Sizey's implementation rather than treating it as a feature ablation.

Tracing and provenance. System-call and Linux audit detail has workload-dependent cost [26]. LPM [27] and CamFlow [28] capture whole-system causal provenance. eBPF executes verified programs at kernel hooks [29]; CAMP uses it more narrowly for task-attributed positive reads and file-backed mmap faults, not complete syscall lineage. Standard and audit lanes therefore make measurement coverage an allocation decision.

Budgeted measurement. Active learning selects labels under measurement limits [30]. CAMP analogously observes peak RSS for every task but consumption only for selected tasks, combining risk/information exploitation with process-aware exploration at exact budgets. Nonuniform selection can bias history: the random lane records inclusion probabilities and applies capped Horvitz–Thompson weights [31]. This reduces bias for stochastic exploration but does not make deterministic risk/information selections a probability sample; CAMP therefore claims selection yield, not an unbiased complete audit set.

VII. Conclusion

CAMP demonstrates that workflow memory allocation is governed by the granularity of available evidence, not model selection. Peak memory anomalies reside in heavy tails driven by actual runtime data consumption, which cannot be inferred from static prelaunch metadata. When continuous observation incurs a system overhead, resource allocation must target areas of highest predictive uncertainty rather than uniform random sampling. Our results [10] confirm this framework pays: selective auditing captures the tail safety of full instrumentation at a fraction of the performance cost, relying on structured historical empirical evidence to safely reduce cluster over-provisioning at scale.

References

[1] A. B. Yoo, M. A. Jette, and M. Grondona, "SLURM: Simple Linux utility for resource management," in *Job Scheduling Strategies for Parallel Processing (JSSPP 2003)*, ser. Lecture Notes in Computer Science, D. Feitelson, L. Rudolph, and U. Schwiegelshohn, Eds., vol. 2862. Springer, 2003, pp. 44–60.

[2] D. Thain, T. Tannenbaum, and M. Livny, "Distributed computing in practice: The Condor experience," *Concurrency and Computation: Practice and Experience*, vol. 17, no. 2–4, pp. 323–356, 2005.

[3] B. Burns, B. Grant, D. Oppenheimer, E. Brewer, and J. Wilkes, "Borg, Omega, and Kubernetes," *Communications of the ACM*, vol. 59, no. 5, pp. 50–57, 2016.

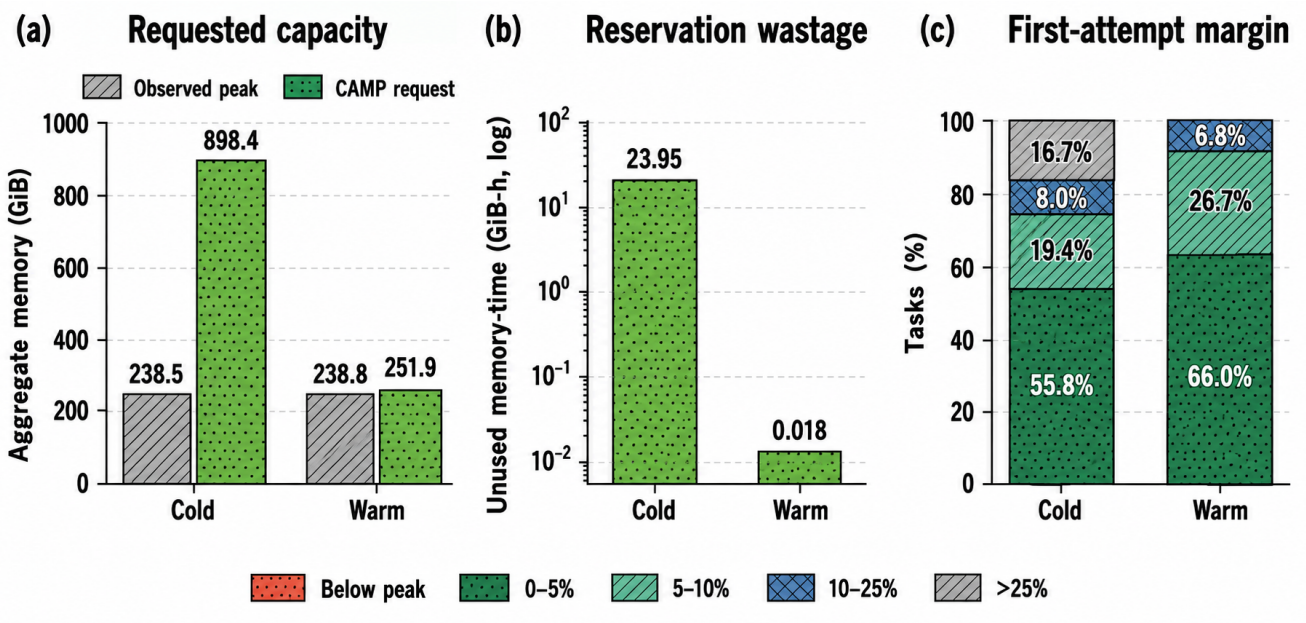


Fig. 5. RQ5 live 6,000-task Bowtie2 cold and fresh warm runs: (a) aggregate observed peak and first-requested memory; (b) runtime-weighted unused memory-time; and (c) first-attempt allocation margin relative to observed peak RSS.

[4] C. Witt, D. Wagner, and U. Leser, "Feedback-based resource allocation for batch scheduling of scientific workflows," in *2019 International Conference on High Performance Computing and Simulation (HPCS)*. Dublin, Ireland: IEEE, 2019, pp. 761–768.

[5] F. Lehmann, J. Bader, N. De Mecquenem, X. Wang, V. Bountris, F. Friederici, U. Leser, and L. Thamsen, "Ponder: Online prediction of task memory requirements for scientific workflows," in *2024 IEEE 20th International Conference on e-Science (e-Science)*. Osaka, Japan: IEEE, 2024, pp. 1–10.

[6] J. Bader, F. Skalski, F. Lehmann, D. Scheinert, J. Will, L. Thamsen, and O. Kao, "Sizey: Memory-efficient execution of scientific workflow tasks," in *2024 IEEE International Conference on Cluster Computing (CLUSTER)*. Kobe, Japan: IEEE, 2024, pp. 370–381.

[7] E. R. Rodrigues, R. L. F. Cunha, M. A. S. Netto, and M. Spriggs, "Helping HPC users specify job memory requirements via machine learning," in *Proceedings of the Third International Workshop on HPC User Support Tools (HUST '16)*. Salt Lake City, UT, USA: IEEE Press, 2016, pp. 6–13.

[8] B. Tovar and et. al., "A job sizing strategy for high-throughput scientific workflows," *IEEE Transactions on Parallel and Distributed Systems*, vol. 29, no. 2, pp. 240–253, 2018.

[9] N. Kandimalla, B. P. Vangala, and T. Malik, "CAMP: Consumption-aware memory prediction for scientific workflows—extended technical report," University of Missouri–Columbia, Tech. Rep., 2026. [Online]. Available: \url{https://radiant-systems-lab.github.io/pdfs/pdfs/pubs/IEEE_SCWorks_26.pdf}

[10] N. Kandimalla and T. Malik, "CAMP experimental artifacts and reproduction package," https://anonymous.4open.science/r/IEEE_WORKS_26_ARTIFACT-B98F/README.md, 2026, accessed: 2026-08-08.

[11] P. Di Tommaso, M. Chatzou, E. W. Floden, P. P. Barja, E. Palumbo, and C. Notredame, "Nextflow enables reproducible computational workflows," *Nature Biotechnology*, vol. 35, no. 4, pp. 316–319, 2017.

[12] P. A. Ewels, A. Peltzer, S. Fillinger, H. Patel, J. Alneberg, A. Wilm, M. U. Garcia, P. Di Tommaso, and S. Nahnsen, "The nf-core framework for community-curated bioinformatics pipelines," *Nature Biotechnology*, vol. 38, no. 3, pp. 276–278, 2020.

[13] nf-core community, "nf-core/sarek release 3.6.0," GitHub release: https://github.com/nf-core/sarek/releases/tag/3.6.0, Oct. 2025, accessed: 2026-08-08.

[14] ——, "nf-core/seqinspector release 1.1.0," GitHub release: https://github.com/nf-core/seqinspector/releases/tag/1.1.0, Jul. 2026, accessed: 2026-08-08.

[15] ——, "nf-core/taxprofiler release 2.0.1," GitHub release: https://github.com/nf-core/taxprofiler/releases/tag/2.0.1, Jun. 2026, accessed: 2026-08-08.

[16] ——, "nf-core/viralmetagenome release 1.1.3," GitHub release: https://github.com/nf-core/viralmetagenome/releases/tag/1.1.3, Jul. 2026, accessed: 2026-08-08.

[17] J. Bader, K. West, S. Becker, S. Kulagina, F. Lehmann, L. Thamsen, H. Meyerhenke, and O. Kao, "Predicting the performance of scientific workflow tasks for cluster resource management: An overview of the state of the art," *arXiv preprint arXiv:2504.20867*, 2025.

[18] R. F. Da Silva, G. Juve, E. Deelman, T. Glatard, F. Desprez, D. Thain, B. Tovar, and M. Livny, "Toward fine-grained online task characteristics estimation in scientific workflows," in *Proceedings of the 8th Workshop on Workflows in Support of Large-Scale Science (WORKS '13)*. New York, NY, USA: ACM, 2013, pp. 58–67.

[19] R. F. Da Silva, G. Juve, M. Rynge, E. Deelman, and M. Livny, "Online task resource consumption prediction for scientific workflows," *Parallel Processing Letters*, vol. 25, no. 3, 2015, art. no. 1541003.

[20] B. Tovar, B. Lyons, K. Mohrman, B. Sly-Delgado, K. Lannon, and D. Thain, "Dynamic task shaping for high throughput data analysis applications in high energy physics," in *2022 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*. IEEE, 2022, pp. 346–356.

[21] C. Witt, J. van Santen, and U. Leser, "Learning low-wastage memory allocations for scientific workflows at IceCube," in *2019 International Conference on High Performance Computing and Simulation (HPCS)*. IEEE, 2019, pp. 233–240.

[22] J. Bader, N. Diedrich, L. Thamsen, and O. Kao, "Predicting dynamic memory requirements for scientific workflow tasks," in *2023 IEEE International Conference on Big Data (BigData)*. IEEE, 2023, pp. 182–189.

[23] J. Bader, N. Zunker, S. Becker, and O. Kao, "Leveraging reinforcement learning for task resource allocation in scientific workflows," in *2022 IEEE International Conference on Big Data (BigData)*. IEEE, 2022, pp. 3714–3719.

[24] J. Bader, F. Lehmann, L. Thamsen, U. Leser, and O. Kao, "Lotaru: Locally predicting workflow task runtimes for resource management on heterogeneous infrastructures," *Future Generation Computer Systems*, vol. 150, pp. 171–185, 2024.

[25] M. Tanash, B. Dunn, D. Andresen, W. Hsu, H. Yang, and A. Okanlawon, "Improving HPC system performance by predicting job resources via supervised machine learning," in *Proceedings of the Practice and*

Experience in Advanced Research Computing on Rise of the Machines (Learning) (PEARC '19). New York, NY, USA: ACM, 2019, pp. 1–8.

[26] L. Zeng, Y. Xiao, and H. Chen, "Linux auditing: Overhead and adaptation," in *2015 IEEE International Conference on Communications (ICC)*. London, UK: IEEE, 2015, pp. 7168–7173.

[27] A. Bates, D. J. Tian, K. R. B. Butler, and T. Moyer, "Trustworthy whole-system provenance for the Linux kernel," in *24th USENIX Security Symposium (USENIX Security 15)*. Washington, D.C., USA: USENIX Association, 2015, pp. 319–334.

[28] T. Pasquier, X. Han, M. Goldstein, T. Moyer, D. Eyers, M. Seltzer, and J. Bacon, "Practical whole-system provenance capture," in *Proceedings of the 2017 Symposium on Cloud Computing (SoCC '17)*. Santa Clara, CA, USA: ACM, 2017, pp. 405–418.

[29] B. Gbadamosi, L. Leonardi, T. Pulls, T. Høiland-Jørgensen, S. Ferlin-Reiter, S. Sorce, and A. Brunström, "The eBPF runtime in the Linux kernel," *arXiv preprint arXiv:2410.00026*, 2024.

[30] B. Settles, "Active learning literature survey," University of Wisconsin–Madison, Computer Sciences Technical Report 1648, 2009.

[31] D. G. Horvitz and D. J. Thompson, "A generalization of sampling without replacement from a finite universe," *Journal of the American Statistical Association*, vol. 47, no. 260, pp. 663–685, 1952.